Jetpack Compose: elementos basicos y usos

Que es Jetpack Compose

- Toolkit declarativo para construir UI en Android con Kotlin.
- La UI se describe con funciones `@Composable` .
- El estado dirige la UI: si el estado cambia, la UI se recompone.

Elementos basicos

1. `@Composable`

- Funcion que describe una parte de la UI.
- Debe ser pura: no hace trabajo pesado ni acceso directo a datos.
- Se usa como si fuera un componente.
  Ejemplo:

```
@Composable
fun Saludo() {
         Text("Hola, clase")
}
```

2. `Text`

- Muestra texto.
- Uso: titulos, etiquetas, mensajes breves.
  Ejemplo:

```
Text(text = "Alumnos")
```

3. `Column` , `Row` , `Box`

- Layouts base para organizar componentes.
- `Column` : vertical, `Row` : horizontal, `Box` : superposicion.
  Ejemplo:

```
Column {
        Text("Nombre")
        Text("Curso")
}
```

4. `Modifier`

- Cadena de modificadores para estilo y comportamiento.
- Uso comun: `padding` , `fillMaxWidth` , `background` , `clickable` .
  Ejemplo:

```
Text(
        text = "Ficha",
        modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp)
)
```

5. `Image`

- Muestra imagenes locales o remotas.
- Se combina con `contentScale` y `modifier` para ajustar tamanio.
  Ejemplo:

```
Image(
        painter = painterResource(id = R.drawable.ic_launcher_foreground),
        contentDescription = "Logo",
        modifier = Modifier.size(48.dp)
)
```

6. `Button` , `IconButton`

- Componentes interactivos.
- Uso: acciones primarias, navegacion, confirmaciones.
  Ejemplo:

```
Button(onClick = { /* accion */ }) {
        Text("Guardar")
}
```

7. `TextField`

- Entrada de texto.
- Se usa con estado para guardar el valor.
  Ejemplo:

```
var nombre by remember { mutableStateOf("") }
TextField(
        value = nombre,
        onValueChange = { nombre = it },
        label = { Text("Nombre") }
)
```

8. `Card`

- Contenedor con elevacion y esquinas redondeadas.
- Uso: listas de elementos, fichas, resumenes.
  Ejemplo:

```
Card(modifier = Modifier.fillMaxWidth()) {
        Text("Alumno: Ana", modifier = Modifier.padding(12.dp))
}
```

9. `LazyColumn` y `LazyRow`

- Listas eficientes.
- Uso: mostrar colecciones grandes con scroll.
  Ejemplo:

```
LazyColumn {
        items(alumnos) { alumno ->
                Text(alumno.nombre)
        }
}
```

10. `Spacer`

- Espacio entre elementos.
- Uso: separar componentes sin usar padding excesivo.
  Ejemplo:

```
Spacer(modifier = Modifier.height(12.dp))
```

Estado y recomposicion

- Estado local con `remember` y `mutableStateOf`.
- Estado compartido con `ViewModel` y `StateFlow`.
- Evitar estado mutable en la vista sin control.
  Ejemplo:

```
val uiState by viewModel.uiState.collectAsState()
Text(uiState.titulo)
```

Navegacion basica

- Usar `Navigation` de Compose para cambiar pantallas.
- Mantener rutas simples y consistentes.
  Ejemplo:

```
NavHost(navController, startDestination = "lista") {
        composable("lista") { ListaClase() }
        composable("detalle") { DetalleAlumno() }
}
```

Theming y estilo

- Tema en `ui/theme` con colores, tipografia y formas.
- Preferir estilos consistentes para toda la app.
- Puedes generar un tema con `MaterialTheme` y personalizarlo aquí.
- Si quieres que el tema siempre se aplique recuerda cambiar el dinamicsColor a false en el tema.
  Ejemplo:

```
MaterialTheme {
        Text("Titulo", style = MaterialTheme.typography.titleLarge)
}
```

Separacion entre componentes y paginas

- Pagina: composable que representa una pantalla completa y coordina estado, eventos y layout principal.

- Componente: pieza reutilizable y pequena que solo recibe datos y callbacks.
- Las paginas viven en `vista/pagina` y los componentes en `vista/componente` .
- Una pagina debe delegar UI repetida a componentes para simplificar la lectura. Ejemplo:

```
@Composable
fun ListaClasePagina(uiState: ListaClaseUIState, onClick: (String) -> Unit) {
        LazyColumn {
                items(uiState.alumnos) { alumno ->
                        TarjetaAlumno(alumno = alumno, onClick = onClick)
                }
        }
}

@Composable
fun TarjetaAlumno(alumno: EstudianteDTO, onClick: (String) -> Unit) {
        Card(modifier = Modifier.fillMaxWidth().padding(8.dp)) {
                Text(alumno.nombre, modifier = Modifier.padding(12.dp))
        }
}
```

Buenas practicas

- Composables pequenos y reutilizables.
- Separar componentes de pagina y componentes reutilizables.
- UDF: la vista solo consume estado y envia eventos.
- No mezclar logica de negocio en la UI.